Republic of Tunisia
The Ministry of Higher Education
and Scientific Research

The University of Carthage

The Higher Institute of Information and
Communication Technologies

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the
Bachelor Degree in Computer Science

Field of Study : Software Engineering and Information Systems

Design and Implementation of a Network Intrusion Detection System

By

Amira BOUAFIF & Hajer JEMAA

Conducted within ITXTREE

Publicly defended on June 6, 2026 in front of the jury members:

Chairman:	Name SURNAME, University Relations Leader, IBM
Reporter:	Name SURNAME, Teacher, ISTIC
Examiner:	Name SURNAME, Teacher, ISTIC
Professional Supervisor:	Firas ABBESSI, Engineer, ITXTREE
Academic Supervisor:	Wassim ABBESSI, Teacher, ISTIC

Academic Year: 2025-2026

Republic of Tunisia
The Ministry of Higher Education and Scientific Research

The University of Carthage

The Higher Institute of Information and Communication Technologies

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the
Bachelor Degree in Computer Science

Field of Study : Software Engineering and Information Systems

---

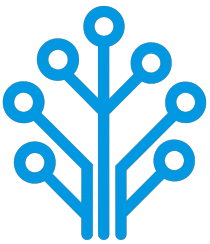
Design and Implementation of a Network Intrusion Detection System

---

By

Amira BOUAFIF & Hajer JEMAA

Conducted within ITXTREE



Authorization of graduation project report submission:

Professional Supervisor:	Academic Supervisor:
Issued on :	Issued on :
Signature:	Signature:

Dedications

I dedicate this End-of-Study project

To my dear parents

Who have supported and encouraged me every step of the way. This is for you, in honor of all the sacrifices you have made for me. I hope to have made you both proud and happy.

To my dear sisters and brothers

Who have been a great support and a source of joy in my life. I am grateful for your love and encouragement throughout this journey.

To all my family and friends

Thank you all for your support and encouragement throughout all this time. I am truly blessed to have you in my life.

Amira BOUAFIF

Dedications

I dedicate this End-of-Study project

To my dear parents

To my dear sisters

To all my family and friends

Hajer JEMAA

Acknowledgments

We would like to extend our sincere gratitude to all those who have supported us throughout the course of this End-of-Study project. Their guidance, encouragement, and support have been invaluable throughout this journey.

First and foremost, we would like to thank our academic supervisor, Professor Wassim ABBESSI, for his guidance and insights, which have been a great help in the successful completion of this project.

Our gratitude also go to ITXTREE and, in particular, our professional supervisor, Mr. Firas ABBESSI, for providing us with the opportunity to work on this project and for their professional mentorship.

We also extend our deep appreciation to the professors at the Higher Institute of Information and Communication Technologies for their invaluable support and teachings, which have greatly contributed to our academic journey.

Finally, we thank everyone who has, in one way or another, helped in making this project a success.

Contents

List of Figures

List of Tables

General Introduction

Chapter 1

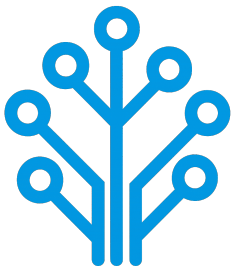
Project Context

1.1 Introduction

This chapter serves as an introduction to the project. We begin with a presentation of the company hosting the internship, a study of existing network intrusion detection systems. Then, we present the problem and the proposed solution. Finally, we end the chapter with a conclusion.

1.2 Presentation of the company

ITXTREE is an early-stage technology startup based in Sidi Rzig, Tunisia, focused on building a solid foundation for future digital solutions it is built on the core values of integrity, quality, curiosity and adaptability.



1.3 Study of the existing

Cybersecurity is a highly sensitive domain where the ability to react quickly can determine the impact of an attack. The earlier a threat is identified, the more effectively its consequences can be limited.

For this reason, Network Intrusion Detection Systems (NIDS) play a central role in monitoring and analyzing network traffic. The most widely used approach relies on:

- **Signature-based:** These systems analyze network traffic, often through deep packet inspection and compares it against a signature database. When a match is found

an alert is triggered. Well-known tools such as Snort [1] and Suricata [2] are representative of this approach.

- **Anomaly-based:** Anomaly detection is learning normal behavior of the network through statistical analysis of traffic parameters. If there is a sudden increase of traffic on a port outside of normal thresholds, it might be an indication of a brand-new threat [1].

1.4 Criticism of the existing

Despite their widespread use, signature-based NIDS present several limitations:

- **High Maintenance and Reactivity:** Their effectiveness is essentially tied to the completeness and accuracy of their signature rule sets. As attack techniques continuously evolve, these systems require frequent updates to remain effective. This maintenance process is both time-consuming and reactive since new signatures can only be created after an attack has already been identified and analyzed.
- **Performance:** As the number of rules increases, the system must process a larger set of comparisons for each packet, which can lead to slower detection. In a context where reaction time is critical, this degradation directly affects the system's ability to respond effectively.
- **Rigidity and Evasion Vulnerability:** Attackers can exploit the rigidity of signature-based detection systems by introducing slight variations in attack patterns, allowing them to evade existing signatures while preserving the same malicious intent. Since many signature rules are publicly available or widely shared with the security community, attackers may analyze them to identify gaps and design variants that bypass detection mechanisms.

Overall, these systems rely on exact matching and predefined logics which limits their adaptability in the face of evolving and increasingly sophisticated threats.

1.5 Proposed solution

With the growing integration of Artificial Intelligence (AI) in security systems, new approaches aim to overcome the limits of traditional methods. To address these challenges, we propose an AI-driven NIDS capable of learning patterns from network traffic and classifying different types of network activity instead of relying solely on predefined rules.

1.6 Project Pipeline

A project pipeline is a structured sequence of stages that outlines the workflow and processes involved in the development and deployment of a project. It serves as a roadmap, guiding the team through various phases, from initial planning to final delivery.

Figure 1.1 illustrates the project pipeline for our network intrusion detection system.

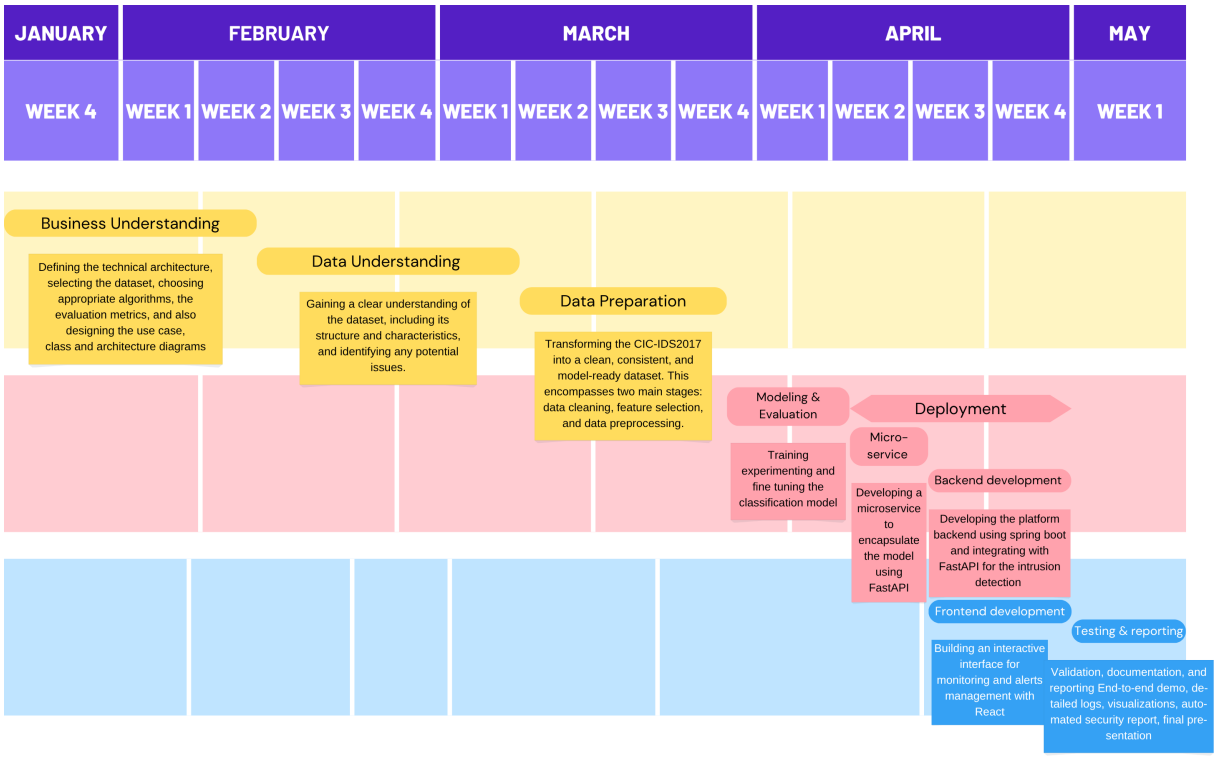


FIGURE 1.1 – Project Pipeline

1.7 Methodologies

In this section, we present the methodologies that we use in this project, including the machine learning methodology CRISP-DM and the software development methodology UML.

1.7.1 CRISP-DM

The CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology is a widely used framework for structuring machine learning projects. It consists of six phases:

- **Business Understanding:** Defining project objectives and requirements and gaining a clear understanding of the business problem.
- **Data Understanding:** Collecting and exploring the data to gain insights and assess its quality by identifying data quality issues.
- **Data Preparation:** Cleaning and preparing the data for modeling by selecting relevant features, handling missing values and splitting the dataset.
- **Modeling:** Selecting and applying appropriate modeling techniques, building, testing and tuning models to optimize their performance.
- **Evaluation:** Evaluating the model's performance and determining if it meets the business objectives.
- **Deployment:** Deploying the model into production, monitoring and maintaining the model's performance.

Figure 1.2 illustrates these phases and their relationships.

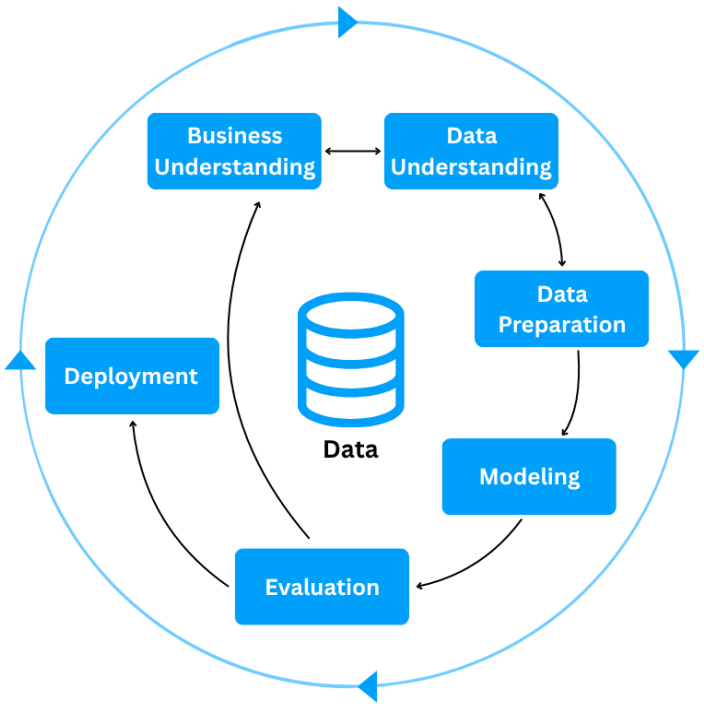


FIGURE 1.2 – CRISP-DM Methodology

1.7.2 UML

For the modeling of the software system, we use UML (Unified Modeling Language), a standardized modeling language used to specify, visualize, construct, and document the artifacts of software systems through diagrams such as use case, class, and sequence diagrams. There are two types of UML diagrams:

- **Behavioral Diagrams:** State machine, activity, use case, sequence, communication and interaction overview diagrams.
- **Structural Diagrams:** Class, object, component, deployment and package diagrams.

UML makes complex systems easier to design and communicate, it provides a common language for developers, architects, and stakeholders [3].

1.8 Conclusion

In this chapter, we introduce the host company ITXTREE, study existing network intrusion detection systems, discuss their shortcomings and finally present a solution. In the next chapter we specify the system requirements.

Chapter 2

Requirements Specification

2.1 Introduction

Throughout this chapter, we specify both the functional and non-functional requirements, identify the system actors, and present the use case diagram. By defining these elements, we aim to establish a clear and shared understanding of the system's expected behavior and constraints, providing a solid foundation for the subsequent design and implementation phases.

2.2 Functional Requirements

Functional requirements define *what* the system should perform. The main functionalities of our system are as follows:

— Authenticate;

— View Dashboard;

— View Statistics summary;

— Trigger Network Analysis;

— View Alerts List;

— View Reports;

2.3 Non-Functional Requirements

Non-functional requirements define *how* the system should perform its functions. They describe the quality attributes and constraints of the system:

- **Performance:** The system must process network traffic and generate analysis results with minimal latency, ensuring fast detection of potential intrusions.
- **Security:** The system must ensure secure authentication and enforce role-based access control, preventing unauthorized actions and protecting system integrity, while supporting non-repudiation to guarantee accountability and traceability of all operations.

- **Maintainability:** The system should be easy to maintain, allowing for efficient debugging, updates, and modifications to existing components.
- **Usability:** The system should provide an intuitive interface, enabling users to efficiently navigate and interact with its functionalities.

2.4 Actors Identification

Actors are external entities, users, organizations, or external systems that interact with the system to achieve one or more functional requirements, with each actor representing a specific role.

In the context of our project, we identify two primary actors, as illustrated in Figure 2.1

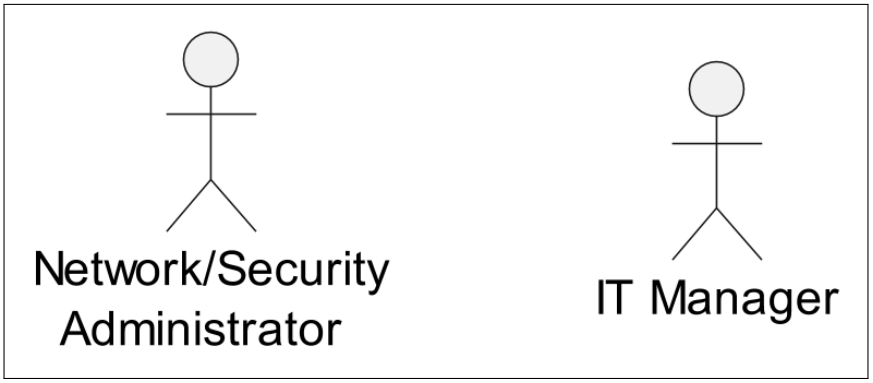


Figure 2.1 – Actors

Table 2.1 presents the roles of each actor in our system.

Table 2.1 – Roles of Actors

Actor	**Role**
Network/Security Administrator	- Authenticate - View Dashboard - View Statistics Summary - Trigger Network Analysis - View Alerts List
It Manager	- Authenticate - View Dashboard - View Statistics Summary - View Reports

2.5 Use Case Diagram

A use case diagram is a visual representation of *how* users interact with the system and *what* a system does through different use cases. Figure 2.2 illustrates that diagram.

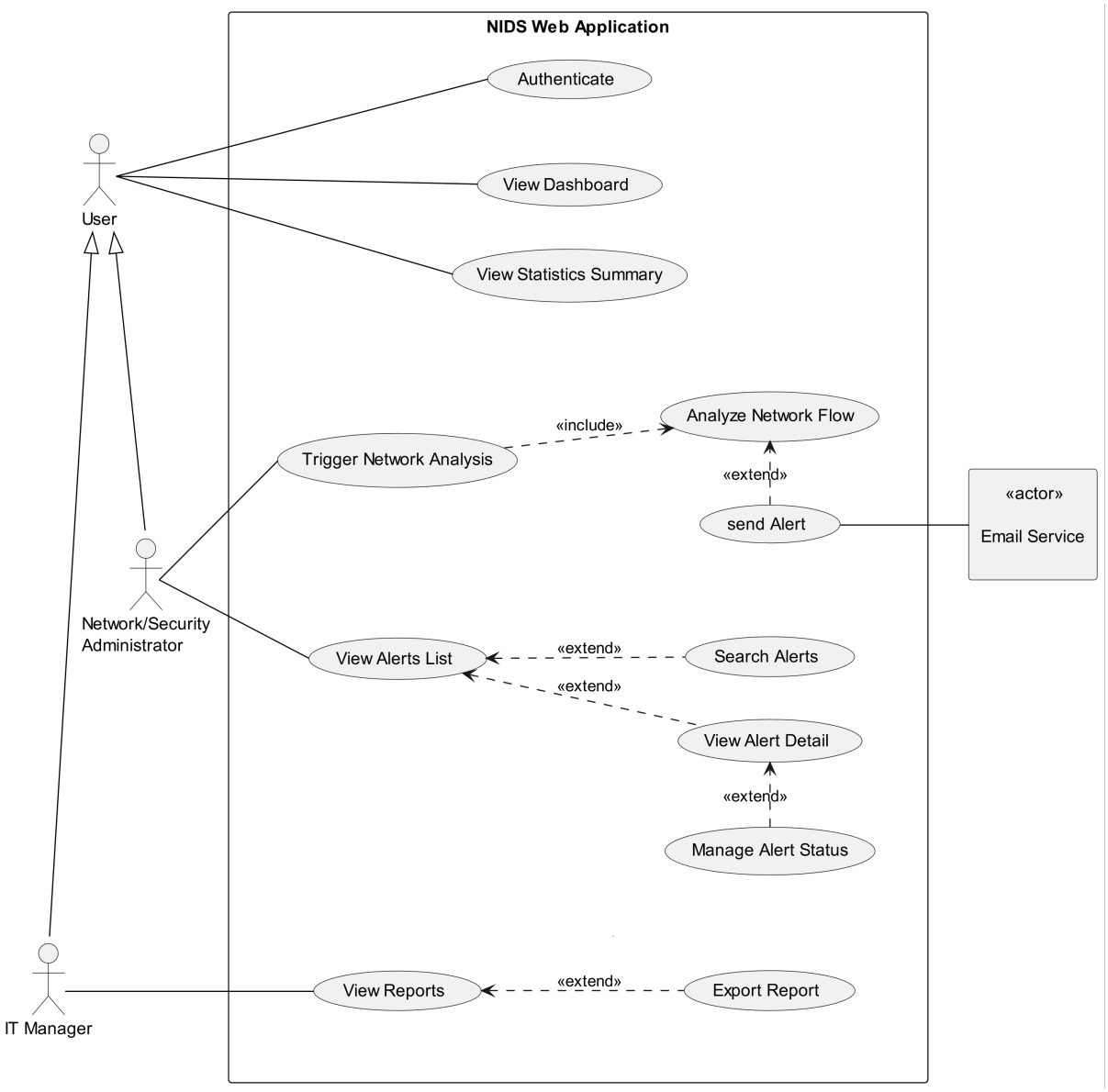


FIGURE 2.2 – Use Case Diagram

2.6 Conclusion

In this chapter, we outline the functional and non-functional requirements of our system, identify the primary actors, and represent the way they interact with the system and the system's functionality via a use case diagram. In the next chapter, we present the conceptual design of the platform.

Chapter 3

Conceptual Design

3.1 Introduction

3.2 Sequence Diagrams

3.3 Class Diagram

3.4 Deployment Diagram

3.5 Conclusion

Chapter 4

Data Collection and Preparation

4.1 Introduction

4.2 Data Collection

4.3 Data Understanding

The Data Understanding phase of the CRISP-DM methodology focuses on loading, describing, and analyzing the dataset to build a solid foundation for the rest of the project. This section dives into the four tasks of this phase: `Initial Exploration`, `Data Description`, `Data Exploration`, and `Data Quality`, with each task targeting an aspect of the data, from its structure and content to its quality.

4.3.1 Initial Exploration

The CIC-IDS2017 dataset is captured over five days, it is distributed across 8 files split by day and session: `Monday`, `Tuesday`, and `Wednesday` each corresponding to a single file, while `Thursday` and `Friday` are split into AM and PM sessions. The `Friday PM` session is further divided into two files to isolate the two attacks, `DDoS` and `PortScan`.

4.3.2 Data Description

As mentioned in the previous subsection, the dataset is split into 8 files as shown in Table 4.4.

Table 4.1 – CIC-IDS2017 dataset files

File	Number of Rows	Number of Columns
Monday	529,918	79
Tuesday	445,909	79
Wednesday	692,703	79
Thursday AM	170,366	79
Thursday PM	288,602	79
Friday AM	191,033	79
Friday PM (DDoS)	225,745	79
Friday PM (PortScan)	286,467	79
Total	2,830,743	—

The dataset contains a total of 2,830,743 rows across the 8 files. All files have consistent columns, which is expected as they are generated by the same CICFlowMeter pipeline. This consistency allows for seamless concatenation of the files into a single DataFrame for the subsequent analysis.

- **Whitespace Issue:** Leading white spaces is found in columns names and are stripped across all files as they can raise `KeyError` exceptions when accessing columns by name.
- **Corrupted Labels:** Corrupted labels are discovered in the Thursday AM file, where three labels (`Web Attack ?  Brute Force`, `Web Attack ?  XSS`, `Web Attack ?  Sql Injection`) contain mojibake character instead of en dash (-), and are therefore corrected.

These fixes are purely cosmetic and do not alter the underlying data values.

- **Feature Description:** The dataset contains 79 columns, of them 78 are numeric level-flow features and the last one is the target Label. These features can be grouped into 12 categories as shown in Table 4.2.

TABLE 4.2 – Feature Groups

Group	Features	Description
Flow Metadata	`Destination Port`, `Flow Duration`	Total duration of the flow and the destination port
Packet/Byte Count	`Total Fwd Packets`, `Total Backward Packets`, `Total Length of Fwd Packets`, `Total Length of Bwd Packets`, `act_data_pkt_fwd`	Number of packets and packet bytes in both forward (from source to destination) and backward directions (from destination to source), and packets having actual data in the forward direction.
Packet/Segment Length Statistics	`Fwd Packet Length Max/Min/Mean/Std`, `Bwd Packet Length Max/Min/Mean/Std`, `Max/Min Packet Length`, `Packet Length Mean/Std/Variance`, `Average Packet Size`, `Avg Fwd Segment Size`, `Avg Bwd Segment Size`	Packets and segments length statistics (e.g. max, min, mean, std etc...).
Flow Rate	`Flow Bytes/s`, `Flow Packets/s`, `Fwd Packets/s`, `Bwd Packets/s`, `Down/Up Ratio`	Flow throughput, forward and backward packet rates, flow asymmetry ratio (backward packets (download) to forward packets (upload)).
Inter-Arrival Time	`Flow IAT Max/Min/Mean/Std`, `Fwd IAT Max/Min/Mean/Std/Total`, `Bwd IAT Max/Min/Mean/Std/Total`	Time between consecutive packets in the flow, forward, and backward directions.
TCP Flag Count	`FIN Flag Count`, `SYN Flag Count`, `RST Flag Count`, `PSH Flag Count`, `ACK Flag Count`, `URG Flag Count`, `CWE Flag Count`, `ECE Flag Count`, `Fwd PSH Flags`, `Bwd PSH Flags`, `Fwd URG Flags`, `Bwd URG Flags`	TCP flags counts for the entire flow, forward and backward directions.
Header Length	`Fwd Header Length`, `Bwd Header Length`, `Fwd Header Length.1`	Backward and forward header lengths.
Bulk Transfer Statistics	`Fwd Avg Bytes/Bulk`, `Fwd Avg Packets/Bulk`, `Fwd Avg Bulk Rate`, `Bwd Avg Bytes/Bulk`, `Bwd Avg Packets/Bulk`, `Bwd Avg Bulk Rate`	Average forward and backward bulk (a sequence of consecutive packets in the same direction) transfer statistics.

Subflow Packet/Byte Count	`Subflow Fwd Packets`, `Subflow Fwd Bytes`, `Subflow Bwd Packets`, `Subflow Bwd Bytes`	Number of packets and packet bytes in both directions in a subflow.
Window/Segment Size	`Init_Win_bytes_forward`, `Init_Win_bytes_backward`, `min_seg_size_forward`	Window sizes in both the forward and backward directions and the minimal segment size in the forward direction.
Active/Idle Time	`Active Max/Min/Mean/Std`, `Idle Max/Min/Mean/Std`	Duration of active and idle periods within a flow.
Target	`Label`	The target label indicating the class the flow belongs to.

- **Duplicate Column:** The column `Fwd Header Length.1` is identified as a duplicate of `Fwd Header Length`, confirmed by the fact that both contain the same values.
- `Packet Length Mean` and `Average Packet Size` both measure the average packet size but are calculated differently: the former uses the statistical mean, while the latter divides the total length of packets by the total number of packets[4]. The two do not hold the same values.

All 54 features are of type `int64`, 24 features are of type `float64`, with the last feature, the target *Label*, being of type `object`.

- **Class Distribution:** The dataset contains 15 classes, BENIGN and 14 attacks, Table 4.3 shows the distribution of these classes across the dataset files.

Table 4.3 – Class Distribution

File	Classes	Class Count
Monday	BENIGN	529,918
Tuesday	BENIGN	432,074
	FTP-Patator	7,938
	SSH-Patator	5,897
Wednesday	BENIGN	440,031
	DoS Hulk	232,443
	DoS GoldenEye	10,293
	DoS Slowloris	5,796
	DoS Slowhttptest	5,499
	Heartbleed	11
Thursday AM	BENIGN	168,186
	Web Attack - Brute Force	1,507
	Web Attack - XSS	652
	Web Attack - Sql Injection	21
Thursday PM	BENIGN	288,566
	Infiltration	36
Friday AM	BENIGN	189,067
	Bot	1,966

Friday PM (DDoS)	DDoS	128,027
	BENIGN	97,718
Friday PM (PortScan)	PortScan	158,930
	BENIGN	127,537

With the exception of Monday, which contains only BENIGN traffic, all files contain at least one attack class alongside BENIGN samples. Each attack class is present in only one file.

Table 4.4 – CIC-IDS2017 dataset classes

No	Class	Description
1	BENIGN	Normal traffic.
2	Dos Hulk	A denial-of-service attack using Http Unbearable Load King (Hulk) tool used to overwhelm web servers by generating a large number of unique randomized requests.
3	PortScan	
4	DDoS	
5	Dos GoldenEye	
6	FTP-Patator	
7	SSH-Patator	
8	DoS slowloris	
9	DoS Slowhttptest	
10	Bot	
11	Web Attack - Brute Force	
12	Web Attack - XSS	
13	Infiltration	
14	Web Attack - Sql Injection	
15	Heartbleed	

4.3.3 Data Exploration

4.4 Data Preparation

4.5 Data Augmentation

4.6 Conclusion

Chapter 5

Modeling

5.1 Introduction

5.2 State of the Art

5.3 Adopted Approach

5.4 Training and Hyperparameter Optimization

5.5 Conclusion

Chapter 6

Realization

6.1 Introduction

6.2 Development Tools and Technologies

6.3 Performance Analysis and Evaluation Metrics

6.4 User Interfaces

6.5 Integration

6.6 Conclusion

General Conclusion

Webography

[1] https://www.snort.org/. Accessed on: April 6, 2026.

[2] https://suricata.io/. Accessed on: April 6, 2026.

[3] https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/. Accessed on: April 2, 2026.

[4] https://github.com/ahlashkari/CICFlowMeter. Accessed on: April 23, 2026.

Bibliography

[1] Chris Jackson. *Network Security Auditing*. 800 East 96th Street, Indianapolis, IN 46240 USA: Cisco Press, 2010. ISBN: 978-1-58705-352-8.